Steps to create JIRA configuration for Workflow

We don't need a paid JIRA account to test this properly.

We can do it on an individual basis using **Jira Cloud Free**.

Step by Step.

Step 1 : Create a Free Jira Cloud Instance

1. Go to: https://www.atlassian.com/software/jira/free
2. Click **Get it free**
3. Sign up with your existing email (you can use the same one you use for GitHub if you want).
4. Choose:
    - Product → **Jira Software**
    - Plan → **Free**
    - Team size → anything (doesn't matter)
5. Create a new site. You'll get something like:

https://your-company-name.atlassian.net
That URL is important.

Step 2 : Create a Test Project

Inside Jira:

1. Click **Projects**
2. Create Project
3. Choose:
    - Template → Scrum or Kanban (either is fine)
4. Name it something like:

AI Agent SDLC
Jira will create a project key automatically, for example:
SCRUM-5
Now you can create test tickets like:
SCRUM-4
SCRUM-3
SCRUM-2

Step 3 : Create a Sample Ticket

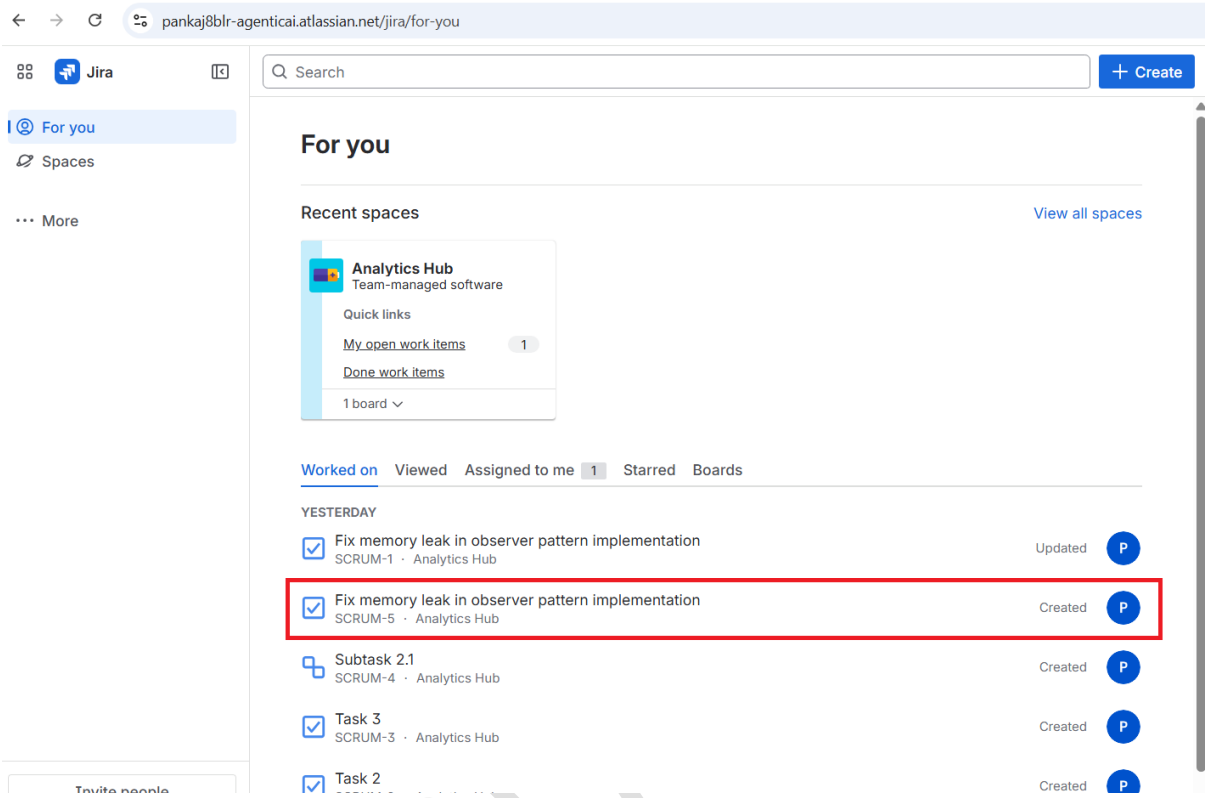
Create an issue with:
Summary:
Fix memory leak in observer pattern implementation
Description:

Pankaj Agrawal

The observer pattern implementation in the event module does not properly detach listeners, causing memory leaks. Update implementation to remove unused observers.

Now you have a real JIRA issue like:

SCRUM-5



Step 4 : Generate API Token for Jira

Jira Cloud uses API token + email authentication.

1. Go to: https://id.atlassian.com/manage-profile/security/api-tokens
2. Click **Create API token**
3. Copy the generated token

Keep it safe.

Step 5 : Add Jira Credentials to .env

Add this to your .env:

```
JIRA_BASE_URL=https://your-company-name.atlassian.net
JIRA_EMAIL=your-email-used-for-jira
JIRA_API_TOKEN=your_generated_token
```

Rebuild Docker:

Pankaj

```
docker compose down
docker compose build --no-cache
docker compose up -d
```

Step 6 : Production-Grade Jira Connector Code

Inside agents/jira_agent.py, use this:

```
import os
import requests
from requests.auth import HTTPBasicAuth

def get_ticket_context(ticket_id):
    base_url = os.getenv("JIRA_BASE_URL")
    email = os.getenv("JIRA_EMAIL")
    api_token = os.getenv("JIRA_API_TOKEN")

    url = f"{base_url}/rest/api/3/issue/{ticket_id}"

    response = requests.get(
        url,
        auth=HTTPBasicAuth(email, api_token),
        headers={"Accept": "application/json"},
    )

    if response.status_code != 200:
        raise Exception(f"Failed to fetch JIRA ticket: {response.text}")

    data = response.json()

    summary = data["fields"]["summary"]
    description = data["fields"]["description"]

    # Safely extract plain text description
    desc_text = ""
    if description:
        for block in description.get("content", []):
            for item in block.get("content", []):
                desc_text += item.get("text", "")

    return f"{summary}\n\n{desc_text}"
```

```
Now restore orchestrator to:

jira_text = get_ticket_context(ticket_id)
```

Step 7 : Test Full Production Flow

Inside container:

```
docker compose exec worker bash
python
```

Then:

```
from agents.orchestrator import run_full_agentic_flow
run_full_agentic_flow("SCRUM-5")
```

What Now Happens in Full Agentic Mode

1. Jira agent pulls real ticket via REST API
2. Impact engine performs semantic search on Qdrant
3. Dependency graph enriches impacted methods
4. LLM proposes changes
5. Git agent creates branch
6. Commit pushed to GitHub

This is now:

Real JIRA
Real GitHub
Real Vector DB
Real Agent Orchestration

No hardcoded strings.

Pankaj Agrawal